Chapitre 5 : Formulaires et événements

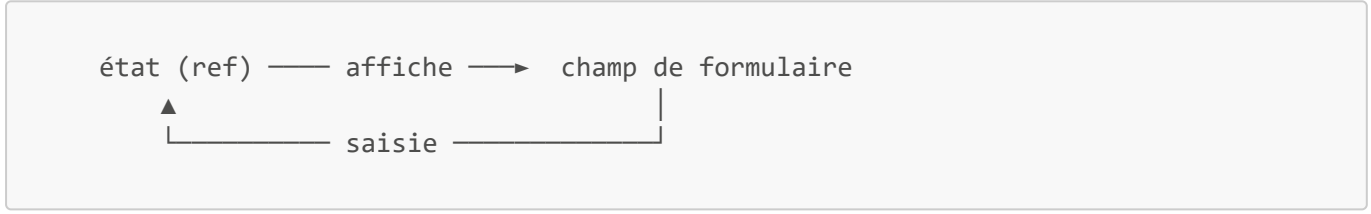
> **Objectifs** : lier des champs de formulaire à l'état avec `v-model`, utiliser les modificateurs, gérer les événements souris/clavier, valider un formulaire côté client. *Composition API uniquement (`<script setup>`).*

> **Comment lire les exemples** : dans les templates, `<!-- → ... -->` indique **ce qui s'affiche ou la valeur obtenue** ; dans le script, `// →` indique le résultat. Chaque exemple est **complet** (script + template) et peut être collé dans un composant d'un projet `create-vue` pour être testé.

---

1. `v-model` sur les différents types de champs

`v-model` crée une **liaison bidirectionnelle** : l'état met à jour le champ, **et** la saisie met à jour l'état.



C'est un raccourci de deux liaisons que l'on pourrait écrire à la main :

```
<script setup>
import { ref } from "vue";
const texte = ref("");
</script>

<template>
  <!-- Version manuelle : :value affiche l'état, @input met l'état à jour -->
  <input :value="texte" @input="texte = $event.target.value">

  <!-- Version avec v-model : strictement équivalente, en une seule directive -->
  <input v-model="texte">

  <p>Vous avez saisi : {{ texte }}</p>
  <!-- → après avoir tapé « Vue » : Vous avez saisi : Vue    (mis à jour à chaque
frappe) -->
</template>
```

`v-model` s'adapte au **type de champ** (texte, case, radio, liste), comme le détaillent les sections suivantes.

1.1 Texte, zone de texte

```
<script setup>
import { ref } from "vue";
```

```
const nom = ref("");        // chaîne vide au départ : le champ est vide
const message = ref("");
</script>

<template>
  <input v-model="nom" placeholder="Votre nom">
  <textarea v-model="message" placeholder="Votre message"></textarea>

  <p>{{ nom }} : {{ message }}</p>
  <!-- Après avoir saisi « Ahmed » et « Bonjour » :
       → Ahmed : Bonjour -->
</template>
```

> ❌ Ne pas écrire `<textarea>{{ message }}</textarea>` : l'interpolation ne fonctionne **pas** dans un `textarea`. Utilisez toujours `v-model`.

L'état initial s'affiche dans le champ : avec `ref("Ahmed")`, l'`input` contient « Ahmed » dès l'ouverture de la page.

1.2 Case à cocher

```
<script setup>
import { ref } from "vue";

const accepte = ref(false);     // une seule case → un booléen
const langages = ref([]);       // plusieurs cases partageant le même v-model → un
tableau
</script>

<template>
  <!-- Une seule case : la valeur est true (cochée) ou false (décochée) -->
  <label><input type="checkbox" v-model="accepte"> J'accepte</label>
  <p>accepte = {{ accepte }}</p>
  <!-- → case décochée : accepte = false
       → case cochée   : accepte = true -->

  <!-- Plusieurs cases liées au MÊME tableau : chaque case coche = sa « value »
est ajoutée -->
  <label><input type="checkbox" value="JS" v-model="langages"> JS</label>
  <label><input type="checkbox" value="Python" v-model="langages"> Python</label>
  <label><input type="checkbox" value="Java" v-model="langages"> Java</label>
  <p>{{ langages }}</p>
  <!-- → aucune case : []
       → JS puis Java cochés : [ "JS", "Java" ]    (l'ordre est celui des clics)
       → on décoche JS : [ "Java" ] -->
</template>
```

1.3 Boutons radio

Les boutons radio partagent le même `v-model` : **un seul** peut être sélectionné, et la valeur est celle de l'attribut `value` du bouton choisi.

```
<script setup>
import { ref } from "vue";
const taille = ref("M");      // valeur initiale : le bouton « M » est
présélectionné
</script>

<template>
  <label><input type="radio" value="S" v-model="taille"> S</label>
  <label><input type="radio" value="M" v-model="taille"> M</label>
  <label><input type="radio" value="L" v-model="taille"> L</label>
  <p>Taille : {{ taille }}</p>
  <!-- → au départ : Taille : M
       → clic sur L : Taille : L -->
</template>
```

1.4 Liste déroulante

```
<script setup>
import { ref } from "vue";

const pays = ref("");           // sélection simple : une chaîne
const villes = ref([]);         // select « multiple » : un tableau
const options = [
  { code: "tn", nom: "Tunisie" },
  { code: "fr", nom: "France" },
  { code: "ma", nom: "Maroc" }
];
</script>

<template>
  <select v-model="pays">
    <!-- Option vide et désactivée : sert de texte d'invite. value="" correspond à
pays = "" -->
    <option disabled value="">Choisir un pays…</option>
    <!-- Une option par élément ; :value lie la valeur, le texte affiché est le
nom -->
    <option v-for="o in options" :key="o.code" :value="o.code">{{ o.nom }}
</option>
  </select>
  <p>Code choisi : {{ pays }}</p>
  <!-- → au départ : Code choisi :        (chaîne vide)
       → on choisit « France » : Code choisi : fr -->

  <!-- multiple : on peut sélectionner plusieurs lignes (Ctrl + clic) → v-model
reçoit un tableau -->
  <select v-model="villes" multiple>
    <option>Tunis</option>
```

```
    <option>Sfax</option>
    <option>Sousse</option>
  </select>
  <p>{{ villes }}</p>
  <!-- → Tunis et Sousse sélectionnées : [ "Tunis", "Sousse" ] -->
</template>
```

> Si l'état initial du `select` ne correspond à **aucune** option, le navigateur affiche une liste vide. D'où l'option `disabled value=""` qui affiche une invite quand `pays` vaut `""`.

1.5 Valeurs non textuelles

Par défaut, `v-model` produit des **chaînes** (ou des booléens pour une case seule). Pour d'autres valeurs :

```
<script setup>
import { ref } from "vue";

const statut = ref("inactif");
const choix = ref(null);
</script>

<template>
  <!-- true-value / false-value : valeurs personnalisées pour une case à cocher --
>
  <input type="checkbox" v-model="statut" true-value="actif" false-
value="inactif">
  <p>{{ statut }}</p>
  <!-- → case décochée : inactif
       → case cochée   : actif   (au lieu de true / false) -->

  <!-- :value (avec deux-points) permet d'associer un objet, un nombre… plutôt
qu'une chaîne -->
  <input type="radio" v-model="choix" :value="{ id: 1 }"> Option 1
  <input type="radio" v-model="choix" :value="{ id: 2 }"> Option 2
  <p>{{ choix }}</p>
  <!-- → après clic sur « Option 2 » : { "id": 2 }    (un VRAI objet, pas une
chaîne) -->
</template>
```

1.6 `v-model` sur un objet de formulaire

Pour un formulaire de plusieurs champs, on regroupe l'état dans **un seul objet**.

```
<script setup>
import { reactive } from "vue";

// Un seul objet pour tout le formulaire
const formulaire = reactive({
  nom: "",
```

```
  email: "",
  age: null,           // null : le champ nombre est vide au départ
  newsletter: false
});
</script>

<template>
  <input v-model="formulaire.nom" placeholder="Nom">
  <input v-model="formulaire.email" type="email" placeholder="Email">
  <!-- type="number" : Vue convertit automatiquement la saisie en nombre -->
  <input v-model="formulaire.age" type="number" placeholder="Âge">
  <label><input v-model="formulaire.newsletter" type="checkbox">
Newsletter</label>

  <pre>{{ formulaire }}</pre>
  <!-- Après avoir saisi Ahmed, ahmed@mail.com, 20 et coché la case :
       → {
           "nom": "Ahmed",
           "email": "ahmed@mail.com",
           "age": 20,              ← un nombre (pas "20")
           "newsletter": true
         } -->
</template>
```

Avantage : pour envoyer le formulaire, on a **un seul objet** à transmettre (`fetch(..., { body: JSON.stringify(formulaire) })`).

---

2. Modificateurs

2.1 Modificateurs de `v-model`

Modificateur	Effet
`.lazy`	Synchronise à l'événement `change` (**perte de focus** ou touche Entrée) plutôt qu'à chaque frappe
`.number`	Convertit la saisie en **nombre** (si la conversion est possible)
`.trim`	Supprime les **espaces** au début et à la fin

```
<script setup>
import { ref } from "vue";

const nom = ref("");
const age = ref("");
const commentaire = ref("");
</script>

<template>
  <input v-model.trim="nom" placeholder="Nom (trim)">
```

```
  <p>[{{ nom }}] — {{ nom.length }} caractère(s)</p>
  <!-- On tape « __Ahmed__ » (les _ représentent des espaces) :
       → [Ahmed] — 5 caractère(s)          (les espaces autour sont supprimés) -->

  <input v-model.number="age" placeholder="Âge (number)">
  <p>{{ age }} — type : {{ typeof age }}</p>
  <!-- On tape « 25 » :   → 25 — type : number     (sans .number : type : string)
       On tape « abc » :  → abc — type : string    (conversion impossible : la
chaîne est conservée) -->

  <input v-model.lazy="commentaire" placeholder="Commentaire (lazy)">
  <p>{{ commentaire }}</p>
  <!-- On tape « Bonjour » : le <p> reste VIDE pendant la frappe,
       puis affiche « Bonjour » au moment où on quitte le champ (ou qu'on appuie
sur Entrée). -->
</template>
```

Les modificateurs se **combinent** : `v-model.trim.lazy="nom"`.

2.2 Modificateurs d'événements

Modificateur	Équivalent JavaScript
`.prevent`	`event.preventDefault()` : annule le comportement par défaut
`.stop`	`event.stopPropagation()` : empêche l'événement de remonter aux parents
`.self`	Ne réagit que si la **cible est l'élément lui-même** (pas un enfant)
`.once`	Ne se déclenche qu'**une seule fois**
`.capture`	Écoute pendant la phase de capture
`.passive`	Écouteur « passif » (améliore les performances du défilement)

```
<script setup>
import { ref } from "vue";

const journal = ref([]);                        // liste de messages pour
visualiser les événements
const log = (message) => journal.value.push(message);

function envoyer() { log("formulaire envoyé"); }
function ouvrir() { log("lien ouvert (sans quitter la page)"); }
function ouvrirCarte() { log("carte ouverte"); }
function supprimer() { log("supprimé"); }
function initialiser() { log("initialisé"); }
</script>

<template>
  <!-- .prevent : le navigateur ne recharge PAS la page à l'envoi -->
  <form @submit.prevent="envoyer">
    <button>Envoyer</button>
```

```
  </form>
  <!-- → clic sur Envoyer : journal = [ "formulaire envoyé" ]
       → sans .prevent, la page serait rechargée et le journal perdu -->

  <a href="https://vuejs.org" @click.prevent="ouvrir">Lien</a>
  <!-- → clic : journal reçoit « lien ouvert (sans quitter la page) » et le
navigateur ne suit pas le lien -->

  <!-- .stop : le clic sur le bouton ne remonte pas jusqu'à la carte -->
  <div class="carte" @click="ouvrirCarte">
    Carte
    <button @click.stop="supprimer">Supprimer</button>
  </div>
  <!-- → clic sur « Carte » (hors bouton) : « carte ouverte »
       → clic sur « Supprimer » avec .stop : « supprimé »  seulement
       → sans .stop : « supprimé » PUIS « carte ouverte » (le clic remonte au
parent) -->

  <!-- .once : le gestionnaire ne s'exécute qu'au premier clic -->
  <button @click.once="initialiser">Initialiser (une fois)</button>
  <!-- → 1er clic : « initialisé » ; les clics suivants ne font rien -->

  <pre>{{ journal }}</pre>
</template>
```

> L'**ordre** des modificateurs compte : `@click.prevent.self` empêche le comportement par défaut de **tous** les clics, alors que `@click.self.prevent` ne l'empêche que pour les clics sur l'élément lui-même.

---

3. Gestion des événements clavier et souris

3.1 Souris

```
<script setup>
import { ref } from "vue";

const x = ref(0);
const y = ref(0);
const survole = ref(false);
const message = ref("");

// event.offsetX / offsetY : position de la souris DANS l'élément (en pixels)
function suivre(event) {
  x.value = event.offsetX;
  y.value = event.offsetY;
}
</script>

<template>
  <div
    class="zone"
```

```
    @mousemove="suivre"
    @mouseenter="survole = true"
    @mouseleave="survole = false"
    @contextmenu.prevent="message = 'clic droit (menu du navigateur bloqué)'"
    @dblclick="message = 'double clic'"
    :style="{ background: survole ? 'lightblue' : 'white' }"
  >
    Position : {{ x }}, {{ y }}
    <!-- → souris au milieu d'une zone de 200 × 100 : Position : 100, 50 -->
  </div>
  <!-- → la zone devient bleue quand la souris entre (mouseenter), blanche quand
elle sort -->
  <p>{{ message }}</p>
  <!-- → après un clic droit : clic droit (menu du navigateur bloqué)
       → après un double clic : double clic -->

  <!-- Modificateurs de bouton et de touche -->
  <button @click.left="message = 'clic gauche'">Gauche</button>
  <button @click.middle="message = 'clic molette'">Molette</button>
  <button @click.ctrl="message = 'Ctrl + clic'">Ctrl + clic</button>
  <!-- → clic simple sur le 3e bouton : rien ; Ctrl maintenu + clic : « Ctrl +
clic » -->
</template>

<style scoped>
.zone { width: 200px; height: 100px; border: 1px solid gray; }
</style>
```

Autres événements : `mousedown`, `mouseup`, `mouseover`, `contextmenu`.

3.2 Clavier

```
<script setup>
import { ref } from "vue";

const saisie = ref("");
const etat = ref("");

const valider = () => (etat.value = `validé : « ${saisie.value} »`);
const annuler = () => { saisie.value = ""; etat.value = "annulé"; };
const sauvegarder = () => (etat.value = "sauvegarde (Ctrl + S)");
</script>

<template>
  <!-- keyup.enter : réagit au relâchement de la touche Entrée (et seulement elle)
-->
  <input v-model="saisie" @keyup.enter="valider" @keyup.esc="annuler"
placeholder="Entrée / Échap">
  <!-- → on tape « Vue » puis Entrée : etat = validé : « Vue »
       → touche Échap : le champ se vide et etat = annulé -->
```

```
  <!-- Combinaison : Ctrl + S ; .prevent bloque la boîte « Enregistrer sous » du
navigateur -->
  <input @keydown.ctrl.s.prevent="sauvegarder" placeholder="Ctrl + S">
  <!-- → Ctrl + S dans ce champ : etat = sauvegarde (Ctrl + S), sans ouvrir la
boîte du navigateur -->

  <p>{{ etat }}</p>
</template>
```

Alias de touches : `.enter`, `.tab`, `.delete` (Suppr et Retour arrière), `.esc`, `.space`, `.up`, `.down`, `.left`, `.right`. Modificateurs système : `.ctrl`, `.alt`, `.shift`, `.meta` (touche Windows / Commande). Pour toute autre touche, on utilise son nom en *kebab-case* (`.page-down`, `.caps-lock`).

> **Quelle touche porte quel nom ?** Ouvrez la console, tapez `window.addEventListener("keydown", e => console.log(e.key))` puis appuyez sur la touche : `e.key` donne son nom (`"PageDown"` devient `.page-down` dans Vue).

> `.exact` : `@click.ctrl.exact` ne réagit que si **seule** la touche Ctrl est enfoncée (pas Ctrl + Maj).

3.3 Accéder à l'événement natif

```
<script setup>
function traiter(event, valeur) {
  console.log(event.target.tagName, valeur);
}
</script>

<template>
  <!-- $event est l'événement natif ; on peut le passer avec d'autres arguments --
>
  <button @click="traiter($event, 5)">A</button>
  <!-- → clic : console affiche « BUTTON 5 » -->

  <!-- Équivalent avec une fonction fléchée -->
  <button @click="(e) => traiter(e, 5)">B</button>
  <!-- → clic : console affiche « BUTTON 5 » -->

  <!-- Sans parenthèses : l'événement est passé automatiquement comme 1er argument
-->
  <button @click="traiter">C</button>
  <!-- → clic : console affiche « BUTTON undefined » (valeur n'est pas fourni) -->
</template>
```

3.4 Un événement global (clavier sur toute la page)

Pour réagir à une touche **n'importe où** dans la page, on écoute `window`. Il faut **retirer l'écouteur** quand le composant disparaît (sinon il reste actif : fuite mémoire, voir le chapitre 7).

```
<script setup>
import { ref, onMounted, onUnmounted } from "vue";

const ouvert = ref(true);

function gererTouche(event) {
  if (event.key === "Escape") {      // « Escape » : nom de la touche Échap
    ouvert.value = false;
  }
}

// onMounted : le composant est dans la page → on installe l'écouteur
onMounted(() => window.addEventListener("keydown", gererTouche));
// onUnmounted : le composant va disparaître → on le retire
onUnmounted(() => window.removeEventListener("keydown", gererTouche));
</script>

<template>
  <div v-if="ouvert" class="fenetre">Appuyez sur Échap pour fermer</div>
  <!-- → au départ : la fenêtre est visible
       → touche Échap, n'importe où dans la page : la fenêtre disparaît -->
</template>
```

---

4. Validation de formulaire côté client

> La validation côté client améliore l'expérience (retour immédiat), mais **ne remplace jamais la validation côté serveur** : un utilisateur peut contourner le navigateur ou appeler l'API directement.

4.1 Validation HTML native

Attributs `required`, `minlength`, `maxlength`, `min`, `max`, `pattern`, `type="email"` : le navigateur bloque la soumission et affiche une bulle.

```
<form>
  <input type="email" required minlength="5">
  <input type="number" min="18" max="99">
  <button>Envoyer</button>
</form>
<!-- → champ email vide + clic sur Envoyer : bulle « Veuillez renseigner ce champ
» (texte du navigateur)
     → « abc » dans l'email : bulle « Incluez un "@" dans l'adresse e-mail » -->
```

Simple, mais le **texte** et le **style** des messages ne se personnalisent pas et varient selon le navigateur. Pour un contrôle complet, on valide avec Vue.

4.2 Validation personnalisée avec Vue

Principe, en trois éléments :

1. un `computed` **`erreurs`** qui calcule les erreurs à partir des valeurs ;
2. un état **`touche`** par champ, pour n'afficher l'erreur **qu'après** que l'utilisateur a quitté le champ (sinon tous les champs seraient en erreur dès l'ouverture) ;
3. un `computed` **`valide`**, vrai quand il n'y a plus d'erreur.

```
<script setup>
import { reactive, computed, ref } from "vue";

// --- ÉTAT ------------------------------------------------------------
const formulaire = reactive({ nom: "", email: "", motDePasse: "", age: null });
// touche[champ] devient true quand l'utilisateur QUITTE le champ (événement blur)
const touche = reactive({ nom: false, email: false, motDePasse: false, age: false
});
const envoye = ref(false);

// --- RÈGLES DE VALIDATION --------------------------------------------
// Retourne un objet : seuls les champs en erreur y figurent, avec leur message
const erreurs = computed(() => {
  const e = {};
  if (!formulaire.nom.trim()) e.nom = "Le nom est obligatoire";
  if (!/^\S+@\S+\.\S+$/.test(formulaire.email)) e.email = "Email invalide";
  if (formulaire.motDePasse.length < 8) e.motDePasse = "8 caractères minimum";
  if (!(formulaire.age >= 18)) e.age = "Vous devez être majeur";   // null >= 18
est faux → erreur
  return e;
});
// Exemple : formulaire vide → erreurs = { nom: "...", email: "...", motDePasse:
"...", age: "..." }
// Exemple : tout est correct → erreurs = {}   (objet vide)

// Valide = aucune clé dans l'objet erreurs
const valide = computed(() => Object.keys(erreurs.value).length === 0);

// --- ACTIONS ---------------------------------------------------------
function soumettre() {
  // On marque tous les champs comme « touchés » pour afficher TOUTES les erreurs
d'un coup
  Object.keys(touche).forEach(champ => (touche[champ] = true));
  if (!valide.value) return;                  // on s'arrête s'il reste des
erreurs
  console.log("Envoi", { ...formulaire });    // → Envoi { nom: "Ahmed", email:
"ahmed@mail.com", ... }
  envoye.value = true;
}

function reinitialiser() {
  Object.assign(formulaire, { nom: "", email: "", motDePasse: "", age: null });
  Object.keys(touche).forEach(champ => (touche[champ] = false));
  envoye.value = false;
}
```

```
</script>

<template>
  <!-- @submit.prevent : on gère l'envoi nous-mêmes ; novalidate : on désactive
les bulles du navigateur -->
  <form @submit.prevent="soumettre" novalidate>
    <div>
      <label for="nom">Nom</label>
      <input id="nom" v-model.trim="formulaire.nom" @blur="touche.nom = true">
      <!-- Le message s'affiche si le champ a été touché ET s'il est en erreur -->
      <small v-if="touche.nom && erreurs.nom" class="erreur">{{ erreurs.nom }}
</small>
    </div>

    <div>
      <label for="email">Email</label>
      <input id="email" type="email" v-model.trim="formulaire.email"
@blur="touche.email = true">
      <small v-if="touche.email && erreurs.email" class="erreur">{{ erreurs.email
}}</small>
    </div>

    <div>
      <label for="mdp">Mot de passe</label>
      <input id="mdp" type="password" v-model="formulaire.motDePasse"
@blur="touche.motDePasse = true">
      <small v-if="touche.motDePasse && erreurs.motDePasse" class="erreur">{{
erreurs.motDePasse }}</small>
    </div>

    <div>
      <label for="age">Âge</label>
      <input id="age" type="number" v-model="formulaire.age" @blur="touche.age =
true">
      <small v-if="touche.age && erreurs.age" class="erreur">{{ erreurs.age }}
</small>
    </div>

    <!-- Désactivé une fois envoyé, pour éviter un double envoi -->
    <button type="submit" :disabled="envoye">S'inscrire</button>
    <!-- type="button" : ce bouton n'envoie PAS le formulaire -->
    <button type="button" @click="reinitialiser">Réinitialiser</button>

    <p v-if="envoye">Inscription envoyée ✓</p>
  </form>
</template>

<style scoped>
.erreur { color: crimson; display: block; }
</style>
```

Déroulement attendu :

Action de l'utilisateur	Résultat à l'écran
Ouverture de la page	Aucun message d'erreur (aucun champ n'est « touché »)
Clic dans « Nom » puis dans « Email » (sans rien saisir)	« Le nom est obligatoire » apparaît sous Nom ; pas encore d'erreur sous Email
Saisie de « abc » dans Email, puis passage au champ suivant	« Email invalide »
Correction en « ahmed@mail.com », puis passage au champ suivant	Le message d'erreur **disparaît** (le `computed` est recalculé)
Clic sur « S'inscrire » avec des champs invalides	**Toutes** les erreurs restantes apparaissent ; rien n'est envoyé
Clic sur « S'inscrire » avec tout valide	« Inscription envoyée ✓ » ; le bouton se désactive ; la console affiche l'objet envoyé
Clic sur « Réinitialiser »	Champs vidés, messages masqués, bouton réactivé

Points d'attention :

- `novalidate` désactive les bulles natives pour laisser la main à notre validation ;
- `@submit.prevent` évite le rechargement de la page ;
- `label for` + `id` : indispensable pour l'accessibilité (clic sur le label, lecteurs d'écran) ;
- `<button type="button">` pour qu'un bouton n'envoie pas le formulaire (le type par défaut d'un bouton dans un `<form>` est `submit`) ;
- `type="number"` : Vue convertit automatiquement en nombre ; un champ vidé donne la chaîne vide `""` (d'où `!(formulaire.age >= 18)`, qui reste vrai dans ce cas).

4.3 Soumission asynchrone

Envoyer le formulaire à un serveur : on gère un état « envoi en cours » et les erreurs réseau.

```
<script setup>
import { reactive, ref } from "vue";

const formulaire = reactive({ nom: "Ahmed", email: "ahmed@mail.com" });
const enCours = ref(false);          // vrai pendant l'envoi
const erreurServeur = ref("");
const succes = ref("");

async function soumettre() {
  enCours.value = true;              // 1. on affiche « Envoi… » et on désactive
le bouton
  erreurServeur.value = "";
  succes.value = "";
  try {
    const reponse = await fetch("https://jsonplaceholder.typicode.com/users", {
      method: "POST",
```

```
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ name: formulaire.nom, email: formulaire.email })
    });
    if (!reponse.ok) throw new Error(`HTTP ${reponse.status}`);   // fetch ne
rejette pas sur 4xx/5xx
    const cree = await reponse.json();
    succes.value = `Inscrit avec l'identifiant ${cree.id}`;
    // → Inscrit avec l'identifiant 11   (cette API de test renvoie toujours id:
11)
  } catch (e) {
    erreurServeur.value = e.message;
    // → HTTP 500 (erreur serveur)  ou  Failed to fetch (réseau coupé / URL
inaccessible)
  } finally {
    enCours.value = false;           // 2. dans TOUS les cas, on réactive le
bouton
  }
}
</script>

<template>
  <form @submit.prevent="soumettre">
    <input v-model="formulaire.nom">
    <input v-model="formulaire.email">
    <button :disabled="enCours">{{ enCours ? "Envoi…" : "S'inscrire" }}</button>
    <!-- → pendant l'envoi : le bouton affiche « Envoi… » et est grisé (évite le
double clic) -->
    <p v-if="succes">{{ succes }}</p>
    <p v-if="erreurServeur" style="color: crimson">Erreur : {{ erreurServeur }}
</p>
  </form>
</template>
```

4.4 Bibliothèques de validation (pour aller plus loin)

Pour des formulaires complexes : **VeeValidate**, **Vuelidate**, ou des schémas **Zod / Yup**. Elles factorisent règles, messages et états (touché, modifié, valide) pour éviter de les réécrire champ par champ.

---

5. Exemple de synthèse : champ de recherche avec raccourcis

```
<script setup>
import { ref, computed } from "vue";

const recherche = ref("");
const villes = ["Djerba", "Tunis", "Sfax", "Médenine", "Gabès", "Zarzis"];

// Liste filtrée : se recalcule à chaque frappe (toLowerCase : on ignore la casse)
const resultats = computed(() =>
  villes.filter(v => v.toLowerCase().includes(recherche.value.toLowerCase())));
// recherche = ""    → les 6 villes (toute chaîne contient "")
```

```
// recherche = "s"   → [ "Tunis", "Sfax", "Gabès", "Zarzis" ]   (contiennent la
lettre s ; Djerba et Médenine non)
// recherche = "ar"  → [ "Zarzis" ]
// recherche = "ne"  → [ "Médenine" ]
// recherche = "xyz" → []
</script>

<template>
  <!-- .trim ignore les espaces autour ; Échap remet recherche à "" (donc affiche
tout) -->
  <input v-model.trim="recherche" @keyup.esc="recherche = ''"
placeholder="Rechercher (Échap pour vider)">

  <ul>
    <li v-for="v in resultats" :key="v">{{ v }}</li>
  </ul>
  <!-- → recherche vide : Djerba, Tunis, Sfax, Médenine, Gabès, Zarzis
       → « s » : Tunis, Sfax, Gabès, Zarzis
       → « ar » : Zarzis
       → « SFAX » (majuscules) : Sfax   (la casse est ignorée) -->

  <p v-if="!resultats.length">Aucun résultat</p>
  <!-- → « xyz » : Aucun résultat -->
</template>
```

> **Les accents comptent** : `includes` compare les caractères exacts. Taper « medenine » ou « gabes » (sans accent) ne trouve **pas** « Médenine » ni « Gabès » : résultat « Aucun résultat ». Pour ignorer les accents, on normalise les deux textes avant de comparer : `texte.normalize("NFD").replace(/[̀-ͯ]/g, "")` (voir l'exercice `slugifier`, chapitre 11).

Résumé

- `v-model` lie un champ à l'état ; son comportement s'adapte au type (texte, case → booléen ou tableau, radio, select).
- Modificateurs `.trim`, `.number`, `.lazy` pour `v-model` ; `.prevent`, `.stop`, `.once`… pour les événements (l'ordre compte).
- Les modificateurs de touches (`.enter`, `.esc`, `.ctrl`) simplifient la gestion du clavier ; un écouteur global se pose dans `onMounted` et se retire dans `onUnmounted`.
- Une validation propre repose sur un `computed` d'erreurs et un état « touché » ; elle ne remplace pas le serveur.
- Pour une soumission asynchrone : état « en cours » avec `try / catch / finally`, bouton désactivé pendant l'envoi.